



Automatic techniques for issue report classification: A systematic mapping study

Muhammad Laiq¹ · Felix Dobsław¹

Received: 25 April 2025 / Accepted: 16 March 2026
© The Author(s) 2026

Abstract

Several studies have evaluated automatic techniques for classifying software issue reports into bugs and non-bugs to assist practitioners in effectively assigning relevant resources based on the type of issue. Currently, no comprehensive overview of this area has been published. A comprehensive overview will help identify future research directions and provide an extensive collection of potentially relevant existing solutions. This study aims to provide a comprehensive overview of the use of automatic techniques to classify issue reports. We conducted a systematic mapping study and identified 46 studies on the topic. The study results indicate that the existing literature applies various techniques for classifying issue reports, including traditional machine learning and deep learning-based techniques, and more advanced large language models. Furthermore, we observe that these studies (a) lack the involvement of practitioners, (b) do not consider other potentially relevant adoption factors beyond prediction accuracy, such as the explainability, scalability, and generalizability of the techniques, and (c) mainly rely on archival data from open-source repositories only. Therefore, future research should focus on real industrial evaluations, consider other potentially relevant adoption factors, and actively involve practitioners.

Keywords Bug report classification · Issue report classification · Software maintenance · Systematic mapping study · Literature review

✉ Muhammad Laiq
muhammad.laiq@miun.se
Felix Dobsław
felix.dobsław@miun.se

¹ Department of Communication, Quality Management and Information Systems, Mid Sweden University, Campus Östersund, Sweden

1 Introduction

Issue management is a complicated and costly process encompassing multiple activities, such as reporting, assigning, and resolving issues (Zou et al. 2018; Parnin and Orso 2011; Laiq et al. 2023). In large-scale software development, a large number of issue reports are submitted by end-users, developers, and testers. For example, Mozilla received 224,408 issue reports in 12 months, and Firefox received 126,967 issue reports in 18 months (Fan et al. 2018). In the next step, these reports are assigned to the relevant developers. Developers then use the submitted information to reproduce the issue, identify root causes, and fix it. This process can be time-consuming and resource-intensive, and can involve back-and-forth discussions among several stakeholders. Moreover, these reports vary significantly in nature. Some reports may be submitted due to misunderstandings, while others might suggest improvements to the documentation or request new features. Some reports describe issues that require a code fix (Antoniol et al. 2008). For example, Kallis et al. (2023) gathered 1,275,881 GitHub issue reports, of which 670,951 (52.6%) were bugs, 472,216 (37%) were feature requests, 76,048 (6%) were questions, and 56,666 (4.4%) were related to documentation.

Consequently, each issue report requires a different action. Thus, practitioners need the ability to identify the type of a newly submitted issue report early in the issue management process to effectively assign the relevant resources. Consider, for example, a developer who is assigned a report flagged as a bug. After several hours of attempting to reproduce the issue, he finds it was actually a feature request mislabeled as a bug. Such false positives¹² waste valuable developer effort and delay the resolution of real bugs. On the scale of hundreds of thousands of reports, these misclassifications accumulate into substantial costs for organizations. Therefore, classifying issue reports into bugs and non-bugs would enable routing only issue reports classified as bugs to developers, saving the time they would otherwise spend on understanding, reproducing, and debugging issue reports that will turn out not to be code-related.

Manually identifying and classifying issue reports is labor-intensive and error-prone (Herzig et al. 2013). Thus, several automatic techniques (e.g., Zanetti et al. 2013; Laiq et al. 2022; Cho et al. 2022; Siddiq and Santos 2022) have been evaluated to assist practitioners in automatically classifying issue reports into bugs (requiring a code correction) and non-bugs (all other categories, e.g., feature requests and questions).

Despite the growing number of studies on automated issue report classification, there is currently no comprehensive overview that organizes and structures this body of work. Such an overview is essential for compiling the various techniques and datasets used in the literature and the evaluation approaches and metrics applied. By systematically mapping this information, researchers and practitioners can better understand the current state of the field, identify gaps and underexplored areas, and observe emerging trends. For example, while many studies focus on improving

¹ https://bugzilla.mozilla.org/show_bug.cgi?id=1392584

² https://bugzilla.mozilla.org/show_bug.cgi?id=1421432

classification accuracy, they often overlook important factors such as the ground truth quality used for training and evaluation. This oversight can significantly affect the reliability of the reported results. A structured perspective is essential to guide future research, support informed decision-making in practice, and ensure that advancements address not only technical performance but also issues of applicability, maintainability, and long-term impact.

In response, we conducted a systematic mapping study (Petersen et al. 2008) to identify the primary studies on the topic, answering the following overarching research question: ***RQ: Which automatic techniques have been proposed in the literature for classifying issue reports?*** Following a systematic approach, we developed a comprehensive map that covers various aspects, including the study's proposed technique, evaluation approach, evaluation metrics, and the issue report features used.

The main contributions of this mapping study are as follows.

- A comprehensive overview/map of the existing work on issue report classification with detailed information about the studies.
- Several insights from the existing literature on issue report classification. For instance, BERT-based models, such as RoBERTa, are recognized as state-of-the-art for this task. Furthermore, traditional ML techniques such as logistic regression and SVM can achieve prediction performance comparable to complex deep learning methods and BERT-based models in classifying issue reports.
- Several potential future research directions for issue report classification. For example, the need for evaluations of the proposed techniques in real industrial settings and a careful assessment of the reliability of ground-truth data used for training and evaluating ML models in issue report classification.

This paper is structured as follows. Section 2 describes the background knowledge and related work on the topic. Section 3 describes the search strategy, selection criteria, data extraction, data analysis and classification, and validity threats to this study. The results are presented in Section 4. Section 5 discusses the overall findings, while Section 6 concludes the paper.

2 Background and related work

Issue management is a costly and complicated process that includes several activities such as reporting, assigning, and resolving issues (Zou et al. 2018; Parnin and Orso 2011; Laiq et al. 2023). In large projects, developers, testers, and end-users submit a large number of issue reports (Fan et al. 2018; Zhang et al. 2015). An issue report³ is an essential software artifact and plays an important role in the debugging and fixing of issues (Zou et al. 2018). To assist practitioners in managing the workload of issue reports and streamlining the issue-fixing processes, significant research in software

³ An issue report is also referred to as defect report, trouble report, bug report, failure report, error report, or incident report.

engineering has focused on automated techniques (Zou et al. 2018) such as issue assignee recommendation (Borg et al. 2024), bug localization (Kim et al. 2013), and duplicate issue report detection (Zhang et al. 2023).

Similarly, to assist practitioners in assigning resources effectively to issue reports, several studies have investigated the use of automatic techniques to categorize issue reports, such as into *bugs* (those that require code correction) and *non-bugs* (e.g., feature requests, questions, and documentation) (Antoniol et al. 2008; Herzig et al. 2013; Zanetti et al. 2013; Colavito et al. 2024). Several automatic techniques have been evaluated in the literature for classifying issue reports into bugs and non-bugs, including traditional ML (Fan et al. 2018; Laiq et al. 2022) and deep learning-based techniques (He et al. 2020; Kwak et al. 2023) and more advanced large language models (Siddiq and Santos 2022; Heo et al. 2024; Laiq et al. 2024).

While a number of primary studies have been conducted on issue report classification, to the best of our knowledge, no comprehensive overview has been conducted. Such an overview would help identify future research directions and provide an extensive collection of relevant existing solutions. Köksal and Öztürk (2022) reviewed the literature on issue report classification and identified 18 primary studies. However, only seven of these studies focus on classifying issue reports into bugs and non-bugs. The findings of their study indicate that traditional ML techniques, including logistic regression, SVM, and random forests, are predominantly used. Only a few studies have explored deep learning-based techniques. The reviewed literature extends up to 2021, and only one study from 2022 is included. This may explain why large language models and advanced deep learning techniques are not included in their analysis. Furthermore, the study does not provide data or insights, nor does it discuss important aspects such as evaluation metrics, evaluation approaches, practitioner involvement, or the quality of ground truth data. The review is not systematic in nature (Kitchenham et al. 2016) as well. In addition, it only reports limited information about primary research, i.e., the techniques used, the number of issue reports, and their sources (i.e., proprietary or open source data). In contrast, our review contributes by providing (a) a comprehensive/map overview of the area following a systematic approach (Petersen et al. 2008), (b) detailed information on primary research, (c) key lessons learned from the existing work, and (d) future research directions.

Moreover, several reviews related to issue management have been conducted in the literature. For example, Zhang et al. (2015) reviewed the literature on issue report analysis. Zou et al. (2018) reviewed the literature on automatic techniques to manage issue reports and conducted a survey with practitioners to gain their insights. Gomes et al. (2019) reviewed the literature on predicting the severity of issue reports. Uddin et al. (2017) reviewed the literature on prioritization of issue reports. While these studies relate to our work, they do not answer the research question posed in this study. Therefore, we conducted a systematic mapping study to answer the posed research question (Petersen et al. 2008).

3 Research method

We conducted a systematic mapping study to answer the research questions listed in Table 1 (Petersen et al. 2008). Figure 1 provides an overview of our research approach.

3.1 Search strategy

Table 2 shows our search string in selected databases to find relevant literature on automatically classifying issue reports. Our search string focuses on studies that classify issue reports according to the classification criteria as defined by Antoniol et al. (2008) and Herzig et al. (2013): (a) those that require code fixes, that is, *bugs*, and (b) *non-bugs*, e.g., documentation, feature requests, and questions. The keywords

Table 1 Research questions

Research question and its sub-research questions
RQ: Which automatic techniques have been proposed in the literature for classifying issue reports? With this research question, we aim to identify proposed automatic techniques for issue report classification. In the following research questions, we further collect information about the pre-processing techniques, the evaluation approaches and metrics, and other related information, such as the context of the study. —RQ1: Which automatic techniques have been proposed? With this question, we aim to provide a collection of applied automatic techniques for classifying issue reports, aiding in understanding the current state of the art. —RQ2: Which features have been used? Features are essential for the success of classification models, as they help identify helpful information from issue reports for classification and inform future feature engineering efforts. —RQ3: Which pre-processing techniques have been used? Like features, pre-processing techniques are key for model performance. Understanding the methods used for feature extraction enables the replication of previous work and emphasizes best practices for preparing issue report data to develop a model. —RQ4: Which evaluation approaches have been used? Evaluation approaches help ensure the reliability and validity of results. Identifying these approaches allows for future studies to adopt appropriate evaluation methods. —RQ5: Which evaluation metrics have been used? Like evaluation approaches, evaluation metrics help ensure the reliability and validity of results. Identifying these metrics allows for future studies to adopt appropriate evaluation metrics. —RQ6: What study contexts have been considered for issue report classification techniques, in terms of study realism and data sources? The realism of a study (open-source vs. closed-source systems) and the source of issue report data (e.g., GitHub, Bugzilla) directly affect the applicability and generalizability of classification techniques. By identifying these aspects, we can determine whether existing techniques have been evaluated in settings that reflect real-world software development and highlight underrepresented contexts that require further research.

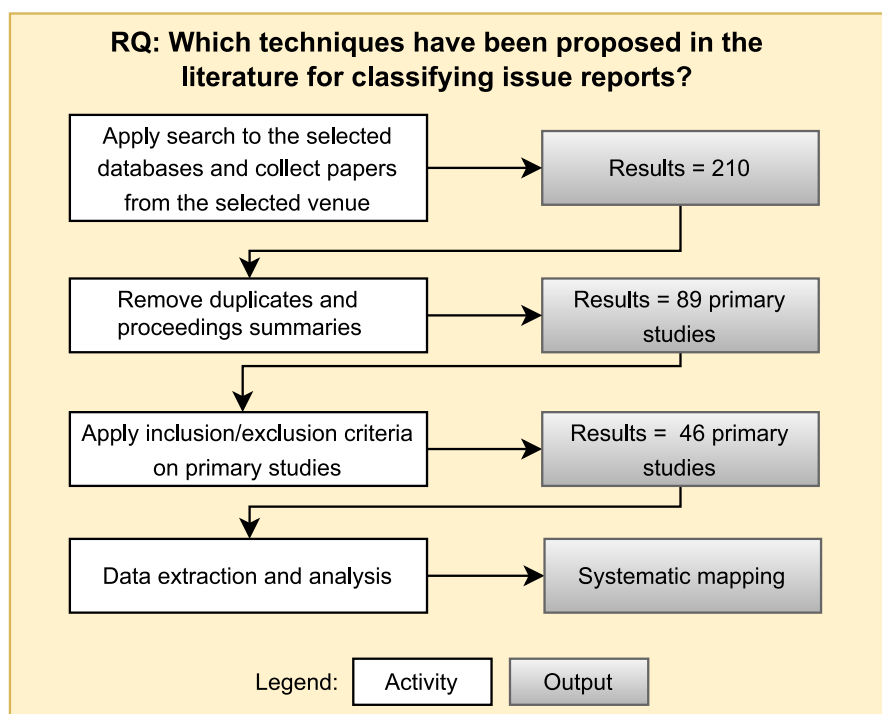


Fig. 1 Systematic mapping workflow

in our search string were defined based on the existing work and our knowledge of this area. For example, 'valid' and 'invalid' terms were also used, as there exists work that uses the terms 'valid' for bugs and 'invalid' for non-bugs. A total of eight articles (Otoom et al. 2019; Colavito et al. 2024; Izadi 2022; Colavito et al. 2023; Laiq et al. 2024; Fan et al. 2018; He et al. 2020; Zanetti et al. 2013) were identified as control articles by the authors of the work, who have expertise in this research area, to assess the adequacy of the search string. The successful retrieval of all eight articles confirms the adequacy of the search string.

In addition to searching relevant papers in the selected databases, we complemented our search strategy by manually collecting articles from a venue⁴ that focuses on issue report classification: the Workshop on NL-based Software Engineering co-located with the International Conference on Software Engineering. We collected papers from all the editions of the workshop from 2022 to 2024.

Table 3 shows the number of search results in each database and the number of collected papers from the selected venue. The search was performed on December 7, 2024.

⁴First edition: <https://nlbse2022.github.io/program/>

Table 2 Search strings in selected databases

Database/Venue	Search string
Scopus	<i>TITLE-ABS-KEY((((“bug report classification” OR “defect report classification” OR “issue report classification”) OR (“classifying bug report” OR “classifying defect report” OR “classifying issue report”) OR ((predict OR determine OR identify) AND (“valid bug reports” OR “invalid bug reports”))))</i>
Web of Science	Search was performed on title, abstract, and keywords and combined using the OR operator: <i>((“bug report classification” OR “defect report classification” OR “issue report classification”) OR (“classifying bug report” OR “classifying defect report” OR “classifying issue report”) OR ((predict OR determine OR identify) AND (“valid bug reports” OR “invalid bug reports”)))</i>
IEEE Xplore	Full-text search was performed using the command feature of IEEE Xplore: <i>((“bug report classification” OR “defect report classification” OR “issue report classification”) OR (“classifying bug report” OR “classifying defect report” OR “classifying issue report”) OR ((predict OR determine OR identify) AND (“valid bug reports” OR “invalid bug reports”)))</i>
ACM	Search was performed on title, abstract, and keywords and manually combined: <i>((“bug report classification” OR “defect report classification” OR “issue report classification”) OR (“classifying bug report” OR “classifying defect report” OR “classifying issue report”) OR ((predict OR determine OR identify) AND (“valid bug reports” OR “invalid bug reports”)))</i>
Selected venue	Papers were collected from the selected venue on issue report classification from 2022 – 2024, i.e., Workshop on NL-based Software Engineering, co-located with the International Conference on Software Engineering.

Table 3 Search results in the selected databases and venue

Database	Search results
Scopus	97
Web of Science	50
IEEE Xplore	39
ACM	13
Selected venue	11
Total = 210	

3.2 Study selection and outcome

We selected papers based on title, abstract reading, and full-text reading. We read full-text only when we could not decide whether to include or exclude a paper based on the title and abstract reading. The following criteria were followed for the study selection:

Inclusion criteria:

I1: The paper is peer-reviewed.

I2: The paper is written in English.

I3: The paper focuses on classifying issue reports according to the classification criteria as defined by Antoniol et al. (2008) and Herzig et al. (2013): (a) those that require code fixes, that is, bugs. (b) non-bugs, e.g., documentation, feature requests, and questions.

Exclusion criteria:

E1: The paper is not peer-reviewed.

E2: The paper is not written in English.

E3: The paper does not address the classification of issue reports into bugs/non-bugs: (a) those that require code fixes, that is, bugs. (b) non-bugs, e.g., documentation, feature requests, and questions.

Figure 1 shows the number of included and excluded papers at each stage. From the initial 210 papers, we identified 109 duplicates and 12 proceedings summaries, leaving 89 unique papers for the selection phase. The first author applied the selection criteria mentioned above to all 89 papers. As a result, 46 papers were selected for data extraction. See an overview in Fig. 1.

In total, 43 papers were excluded according to the exclusion criteria mentioned above. The reasons for exclusion were as follows. Thirty-nine papers were excluded because they did not focus on the classification of issue reports into bugs and non-bugs. Additionally, two papers were not written in English; one was a vision paper that lacked empirical results, and one could not be accessed despite our efforts. The same reviewer, the first author, reconsidered all 43 excluded papers after a gap of seven weeks from the first iteration of selecting papers to reduce the risk of excluding a relevant paper. In addition, the list of the excluded papers with reasons for excluding a paper at this stage is made available at <https://doi.org/10.6084/m9.figshare.31438093> for transparency.

3.3 Data extraction

Table 4 shows the data extraction template we used to extract data from the selected primary studies. The first author performed data extraction on all 46 primary studies. The second author checked the data extraction for ten (approx. 22% of the primary studies) randomly selected primary studies. These studies include papers from diverse years, venues, and study types. There was only one minor discrepancy found between the two authors. Specifically, an *evaluation metric* (Hamming Loss) had not been extracted from a primary study. The main reason for this was that we were mainly interested in the following metrics: Accuracy, Precision, Recall, F-measure,

Table 4 Data extraction template

Item	Description	Mapping to RQs
#1	Meta information, e.g., the title of the paper, publication year, and type of paper, i.e., conference/journal/workshop.	–
#2	Proposed automatic technique, e.g., Logistic regression.	RQ1
#3	Used features with a technique, e.g., title and description of an issue report.	RQ2
#4	Used pre-processing techniques related to feature extraction and embedding, e.g., Word2vec and Term frequency-inverse document frequency (TF-IDF).	RQ3
#6	Used evaluation approach, e.g., 10-fold cross-validation and test on a fixed test set.	RQ4
#5	Used evaluation metrics, e.g., F1-score and accuracy.	RQ5
#7	Study context: (a) study realism: open-source (OSS), closed-source (CSS), and (b) the source of data, e.g., Github and Bugzilla.	RQ6

AUC, and MCC. However, both authors decided to extract all evaluation metrics used in primary studies. Thereafter, all 46 papers were reread to extract any missing evaluation metrics. No other missing metrics were found during this review.

3.4 Analysis and classification

The extracted information was tabulated and visualized using graphs. We also categorized the proposed techniques into higher-level categories, such as traditional ML and deep learning techniques and BERT-based large language models. Similarly, we grouped studies based on evaluation approaches, evaluation metrics, and pre-processing techniques.

The classification scheme for techniques was developed using the extracted data. After completing the data extraction, both authors jointly reviewed the identified techniques and designed an initial classification scheme, grouping techniques into higher-level categories: traditional machine learning techniques (e.g., SVM and Random Forest), deep learning techniques (e.g., CNN and LSTM), and large language models (e.g., BERT-base and RoBERTa). The first author then assigned each study to the agreed categories, and the second author independently reviewed the classifications for consistency and accuracy. A minor discrepancy concerning whether few-shot/zero-shot learning approaches should be grouped under general LLM-based methods was resolved through discussion and consensus, leading to the final agreed-upon categorization. As a result, we placed few-shot and zero-shot learning, T5 models, and BERT-based models in separate categories rather than grouping them under a broader “large language models” category. To support transparency and reproducibility, we have made the complete classification scheme available in the shared dataset: <https://doi.org/10.6084/m9.figshare.31438093>.

3.5 Validity threats

We used the guidelines by Ampatzoglou et al. (2019) to consider the potential validity threats to our study and identify mitigation actions. In the following subsections, we discuss some of the validity threats and our mitigation actions.

3.5.1 Study selection validity

A potential threat to this mapping study is the omission of relevant studies on issue report classification. We mitigated this by carefully developing our search string that focuses on identifying relevant studies on issue report classification. In addition, we have used two indexing (i.e., Scopus and Web of Science) and two publisher-specific databases (i.e., IEEE Xplore and ACM) without restricting our search to papers from specific years to have sufficient coverage. These databases sufficiently cover the software engineering literature (Kitchenham et al. 2016; Usman et al. 2023). Moreover, we also complemented our search strategy by manually identifying relevant studies from a relevant venue dedicated to the topic (see details in Section 3.1).

Another threat to this review is bias during the paper selection process. We mitigated this risk by ensuring that every excluded paper was reconsidered after a seven-week gap by the same reviewer. A similar approach (i.e., test-retest) has been applied and recommended when a single reviewer conducts steps of systematic reviews or mapping studies (Kitchenham 2010; Petersen 2011). The involvement of a single reviewer may still pose a validity threat of missing relevant studies. However, since this work is a systematic mapping study rather than a systematic literature review, the goal is to obtain a good sample of papers instead of exhaustive coverage as recommended in the literature (Kitchenham et al. 2012; Wohlin et al. 2013; Petersen et al. 2015). In this context, the potential risk of excluding a few relevant papers due to a single reviewer does not pose a major threat to the validity of the mapping study. We believe that we have obtained a reasonable sample that includes a diverse set of studies from 2011 to 2024, including publications from top software engineering venues. In addition, we have manually collected all papers on issue report classification from the workshop on topic (NLBSE workshop, see Section 3.1) from 2022 to 2024.

3.5.2 Data validity

Another potential threat to this study is bias in the data extraction and analysis of primary studies. Two reviewers were involved in the data extraction and analysis process to mitigate this threat. The first author performed data extraction on all 46 studies. To increase confidence in data extraction, the second author reviewed a randomly selected subset of ten primary studies to verify the correctness of the extracted data. No discrepancies were found between the two authors (see detailed discussion in Section 3.3).

A limitation of this study is that we did not conduct a quality assessment of the included primary studies. This is not a major threat to the results of this mapping, where we show what techniques have been used for issue report classification. Mapping studies are exploratory (Petersen et al. 2015). Their primary aim is to provide a structured overview of a research area, identify trends, and highlight gaps rather than evaluate the strength of evidence. Thus, quality assessment is not mandatory for them. Therefore, in our study, we did not apply a formal quality assessment, since our objective is to map the techniques that have been applied for the studied task and not to determine their relative effectiveness. However, we acknowledge this as a limitation of our work.

3.5.3 Research validity

The research method chosen for this study is a systematic mapping study based on our aim, i.e., to structure and provide an overview of existing work on issue report classification. Before conducting this review, we developed a review protocol following the systematic mapping guidelines (Petersen et al. 2008; Usman et al. 2023). More than one researcher was involved in the design and execution of the study, and the protocol was iteratively improved before the final execution of the review process.

The data collected in the process of this study have been made available for transparency and replication (including the list of primary studies, the list of papers excluded from the review after full-text reading, and a complete map in the form of a spreadsheet that readers can use to navigate the literature map).

4 Results and analysis

This study provides a comprehensive map of various automatic techniques for issue report classification. The complete map, which is too large to reproduce here, including metadata and other contextual information, can be found at <https://doi.org/10.6084/m9.figshare.31438093>; a simplified version is shown in Table 5. In the complete map, we also provide data to facilitate synthesis; for example, we report the best-performing techniques among the compared techniques as reported by the authors. We also report the top results from each primary study, as well as the classification labels studied in each primary study.

In the following subsections, we first describe the characteristics of the selected primary studies and then present the results of our research questions RQ1–RQ6 stated in Table 1.

Characteristics of the analyzed primary studies: Figure 2 shows the yearly distribution between 2011 and 2024 of primary studies on issue report classification. These studies are published in leading software engineering venues, including the Empirical Software Engineering Journal, the International Conference on Software Engineering, and the Journal of Systems and Software.

Among the included primary studies (46), 16 (34.8%) of the studies belonged to workshops, 15 (32.6%) of the studies belonged to conferences, and another 15 (32.4%) of the studies belonged to journals.

Table 5 A map of automatic techniques for issue report classification. Precision (P), Recall (R), F-score (F1), Accuracy (Acc), Micro Average (Avg), Test on a fixed set (TFS), and 10-fold CV (10FCV)

PS	Technique	Data source	Features	Pre-processing technique	Evaluation strategy	Evaluation metrics
PS1	Rule-based technique	Bugzilla	Title, description	TF-IDF	TFS	P, R
PS2	Random forest, SVM, K-means clustering	Bugzilla, Jira	Summary	TF, TF-IDF, TF-IGM	TFS	R, P, F1
PS3	BERT-based model, RoBERTa, CodeBERT, BERTOverflow, seBERT	GitHub	Title, description	BERT-based tokenizer	TFS	Micro F1
PS4	CNN	GitHub	Image, code,	Embedding layer	TFS	P, R, F1
PS5	KNN	Bugzilla, Redhat Bugzilla	Summary, severity, priority, component, assignee, reporter, OS, reproducibility	TF-IDF	TFS	R, P, F1
PS6	Stacking ensemble, Random forest, SVM, Logistic regression, Multilayer perceptron, SGD, Multinomial Naive Bayes, Naive Bayes	GitHub, Bugzilla, Jira	Title, description	TF-IDF	Stratified 10FCV	Acc, MCC, F1
PS7	SGD	GitHub	Title, body	TF-IDF	TFS	P, R, F1, Avg P, Avg R, Avg F1
PS8	GPT-3.5-turbo base model	GitHub	Title, body	GPT-based tokenizer	TFS	P, R, F1, Avg P, Avg R, Avg F1
PS9	Random tree, SVM, Naive Bayes	GitHub, Bugzilla	Title, description	Manual selection of textual features	5-fold CV	Acc
PS10	Naive Bayes, SVM, KNN, Logistic regression, Decision tree, Random forest	CSS data	Description	TF-IDF	N-fold CV	F1
PS11	RoBERTa	GitHub	Title, body	RoBERTa-based tokenizer	TFS	P, R, F1
PS12	BERT-based model	GitHub	Title, body	BERT-based tokenizer	TFS	P, R, F1, Avg P, Avg R, Avg F1

Table 5 (continued)

PS	Technique	Data source	Features	Pre-processing technique	Evaluation strategy	Evaluation metrics
PS13	Logistic regression, Random forest	Jira	Title, description	N-gram IDF	10FCV, TFS	F1
PS14	Vector space model-based approach	NA	Description	TF-IDF	NA	Acc, P, R
PS15	SVM	Bugzilla	Features based on collaboration graphs	NA	TFS	P, R, F1
PS16	SVM, Random forest	Bugzilla	Title, description, comments, reporter experience, collaboration network, completeness, readability	TF-IDF	Time split with 10FCV	P, R, F1, AUC
PS17	RoBERTa	GitHub	Title, body	RoBERTa-based tokenizer	TFS	P, R, F1, Avg P, Avg R, Avg F1
PS18	BERT-based model	GitHub	Title, description, submitter	BERT-based tokenizer	TFS	P, R, F1
PS19	LSTM	Bugzilla, Jira	Title, description	Skip-gram model	Time split with test on a fixed set	P, R, F1
PS20	Decision tree, Naive Bayes, and Logistic regression.	Jira	Title, description, discussion	LDA	10FCV	F1
PS21	CNN, LSTM	GitHub	Title, body, user manuals	Embedding layer, GloVe, Skip-gram model	5-fold CV	P, R, F1
PS22	Fasttext	GitHub	Title, body	Skip-gram model	10FCV	P, R, F1
PS23	Classification associations rule mining	Bugzilla, Jira	Title, description	Manuall selection of textual features	TFS	Acc
PS24	Naive Bayes, ADTree, Logistic regression	Mantis, Redhat Bugzilla	Summary, severity, priority, component, assignee, reporter, OS, reproducibility	NA	10FCV	P, R, F1
PS25	CNN	Bugzilla	Summary, description	Skip-gram model	Time split with test on a fixed set	P, R, F1, AUC

Table 5 (continued)

PS	Technique	Data source	Features	Pre-processing technique	Evaluation strategy	Evaluation metrics
PS26	KNN, SVM, Logistic regression, Decision tree	CSS data	Heading, description, priority, submitter experience	TF-IDF	Time split with 10FCV	AUC, MCC, P, R, F1
PS27	Distilbert, art-large-mnli	Twitter, Other forum posts	Review description, Metadata-based features, e.g., app rating and category	BERT-based tokenizer	5-fold CV	P, R, F1
PS28	RoBERTa with adapters	GitHub	Title, body	RoBERTa-based tokenizer	TFS	P, R, F1, Avg P, Avg R, Avg F1
PS29	Setfit	GitHub	Title, body	All-mpnet-base-v2	TFS	P, R, F1, Avg P, Avg R, Avg F1
PS30	Naive Bayes, Logistic regression, Decision tree	GitHub	Description	TF-IDF	TFS	Acc, P, R
PS31	BERT-based model	GitHub	Title, body	BERT-based tokenizer	TFS	P, R, F1, Avg P, Avg R, Avg F1
PS32	BERT-based model, XG-Boost, SVM, Logistic regression, SVM, DistillBert, CNN, Random forest	CSS data	Heading, description, priority, submitter experience, completeness	Skip-gram model	Time split with 10FCV	P, R, F1, Acc, AUC, MCC
PS33	CNN	GitHub	Image, code,	Embedding layer	TFS	P, R, F1
PS34	BERT-based model, RoBERTa, ALBERT	GitHub	Title, body, issue-author association	AutoTokenizer from transformers	TFS	P, R, F1, Avg P, Avg R, Avg F1
PS35	Setfit, GPT3.5-turbo	GitHub	Title, body	All-mpnet-base-v2	TFS	P, R, F1, F1-micro
PS36	Feed-forward neural network	GitHub	Title, description, comment	Text-embedding-ada-002	5-fold CV	Acc, P, R, F1
PS37	LSTM, CNN	Bugzilla, Jira	Title, description	NA	10FCV	F-score, AUC, MCC, G-measure
PS38	KNN	Bugzilla	Summary, severity, component, priority	TF-IDF	TFS	Acc

Table 5 (continued)

PS	Technique	Data source	Features	Pre-processing technique	Evaluation strategy	Evaluation metrics
PS39	LSTM	Bugzilla, Jira	Title, body	Skip-gram model	10FCV	AUC, MCC, G-measure, F1
PS40	T5	GitHub	Title, body	T5-based tokenizer	TFS	P, R, F1, Avg P, Avg R, Avg F1
PS41	SGD, Logistic regression, Naive Bayes, Gradient boosting, Decision tree, Random forest, Linear discriminant analysis	Bugzilla	Summary, description	Skip-gram model	TFS	Acc, F1
PS42	SVM, Random forest, Logistic regression, Naive Bayes	GitHub	Title, body	TF-IDF	10FCV	P, R, F1
PS43	BERT-based model	Bugzilla	Title, description	BERT-based tokenizer	TFS	P, R, F1
PS44	XGBoost, Naive Bayes, Logistic regression, Random forest, J48	Bugzilla	Priority, severity, component, text readability, text sentiment	NA	10FCV	P, R, F1
PS45	seBERT	GitHub	Title, body	BERT-based tokenizer	TFS	P, R, F1, Avg P, Avg R, Avg F1
PS46	RoBERTa	GitHub	Title, body	RoBERTa-based tokenizer	TFS	P, R, F1, Avg P, Avg R, Avg F1

4.1 RQ1: Which automatic techniques have been proposed?

Table 6 provides an overview of the proposed automatic techniques in the literature for issue report classification. Traditional ML techniques are predominant, with logistic regression at the top, used in eleven studies, followed by random forest and SVM, used in ten studies each. Naive Bayes has been used in eight studies, while decision trees have been used in five studies. Stochastic gradient boosting and XGBoost have been used in three and two studies, respectively. All other traditional ML techniques have been used in a single study. Deep learning techniques have also been used in several studies, with convolutional neural networks used in six

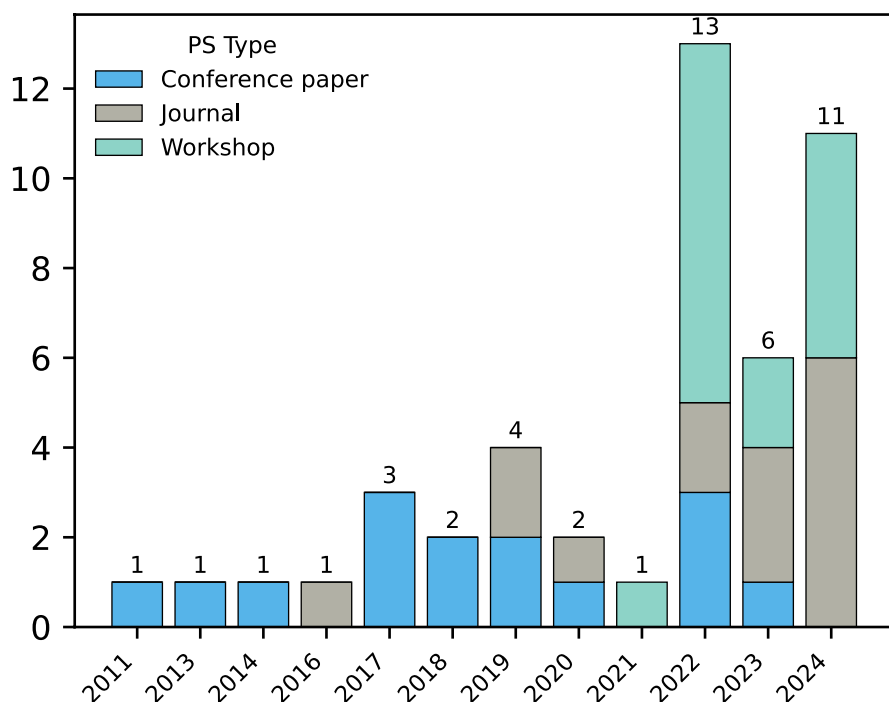


Fig. 2 Yearly distribution of primary studies grouped by their publication venue type

studies and long short-term memory networks in four studies. Transformer-based models, particularly BERT and its variants, are becoming increasingly popular, used in seven studies, followed by RoBERTa, which has been used in five studies, along with other specialized adaptations like seBERT and DistillBERT. Few-shot and zero-shot learning approaches are emerging trends, with five studies evaluating them for issue report classification. The text-to-text transfer transformer (T5) model has been evaluated in a single study.

4.2 RQ2: Which features have been used?

Table 7 shows a summary of the features used in the literature for issue report classification. As shown in the table, the description/body and title/heading of the issue reports are the most used features in the literature, used in 38 and 32 studies, respectively. Other notable features include priority (six studies), severity (six studies), summary (four studies), and component (four studies), each appearing in four to six studies. Features related to submitter experience, collaboration networks, and comments on issue reports were utilized in three studies. All other features were used in one to two studies.

In addition, in the included primary studies, we found that there is generally a lack of analysis to select optimal features (e.g., identifying which features or words lead to misclassification). Only nine of 46 studies conducted such an analysis.

Table 6 Automatic techniques proposed in the literature for classifying issue reports

Category	Technique	Relevant studies	Count
Traditional ML	Logistic regression	PS6, PS10, PS13, PS20, PS24, PS26, PS30, PS32, PS41, PS42, PS44	11
	Random forest	PS2, PS6, PS9, PS10, PS13, PS16, PS32, PS41, PS42, PS44	10
	Support vector machine (SVM)	PS2, PS6, PS9, PS10, PS15, PS16, PS26, PS32, PS32, PS42	10
	Naive Bayes	PS6, PS9, PS10, PS20, PS24, PS30, PS41, PS42, PS44	9
	Decision tree	PS10, PS20, PS26, PS30, PS41	5
	K-nearest neighbors (KNN)	PS5, PS10, PS26, PS38	4
	Stochastic gradient descent (SGD)	PS6, PS7, PS41	3
	XGBoost	PS32, PS44	2
	K-means clustering	PS2	1
	Stacking ensemble	PS6	1
	Fasttext	PS22	1
	ADTree	PS24	1
	Gradient boosting	PS41	1
	Linear discriminant analysis	PS41	1
	J48	PS44	1
Deep learning	Convolutional neural network (CNN)	PS4, PS21, PS25, PS32, PS33, PS37	6
	Long short-term memory network (LSTM)	PS19, PS21, PS37, PS39	4
	Multilayer perceptron	PS6	1
	Feed-forward neural network	PS36	1
BERT-based models	BERT-based model	PS3, PS12, PS18, PS31, PS32, PS34, PS43	7
	RoBERTa	PS3, PS11, PS17, PS34, PS46	5
	seBERT	PS3, PS45	2
	DistillBert	PS27, PS32	2
	CodeBERT	PS3	1
	BERTOverflow	PS3	1
	ALBERT	PS34	1
	Text-to-text transfer transformer	PS40	1
Few shot learning	Setfit	PS29, PS35	2
	GPT-3.5-turbo	PS8	1
Zero shot learning	RoBERTa with adapters	PS28	1
	Bart-large-mnli	PS27	1
	GPT3.5-turbo	PS35	1
Other	Rule based technique	PS1	1
	Vector space model	PS14	1
	Classification associations rule mining	PS23	1

Table 7 Features used with techniques for issue report classification

Feature	Relevant studies	Count
Description/Body	PS1, PS3, PS6, PS7, PS8, PS9, PS10, PS11, PS12, PS13, PS14, PS16, PS17, PS18, PS19, PS20, PS21, PS22, PS23, PS25, PS26, PS27, PS28, PS29, PS30, PS31, PS32, PS34, PS35, PS36, PS37, PS39, PS40, PS41, PS42, PS43, PS45, PS46	38
Title/Heading	PS1, PS3, PS6, PS7, PS8, PS9, PS11, PS12, PS13, PS16, PS17, PS18, PS19, PS20, PS21, PS22, PS23, PS26, PS28, PS29, PS31, PS32, PS34, PS35, PS36, PS37, PS39, PS40, PS42, PS43, PS45, PS46	32
Priority	PS5, PS24, PS26, PS32, PS38, PS44	6
Summary	PS2, PS5, PS24, PS25, PS38, PS41	6
Severity	PS5, PS24, PS38, PS44	4
Component	PS5, PS24, PS38, PS44	4
Submitter	PS5, PS18, PS24	3
Comments	PS16, PS20, PS36	3
Submitter experience, e.g., total submitted issue reports	PS16, PS26, PS32	3
Reproducibility	PS5, PS24	2
Features based on collaboration graphs of issue submitters, e.g., the clusters of highly connected people	PS15, PS16	2
Text completeness	PS16, PS32	2
Text readability	PS16, PS44	2
Assignee	PS5, PS24	2
Operating system	PS5, PS24	2
Code	PS4, PS33	2
Image	PS4, PS33	2
Metadata-based features, e.g., app rating and category	PS27	1
Issue-author association	PS34	1
Text sentiment	PS44	1
User manual	PS21	1

4.3 RQ3: Which pre-processing techniques have been used?

Table 8 presents the reported pre-processing techniques employed for issue report classification across various studies. The table indicates that TF-IDF is the most commonly used approach, appearing in 13 studies. Modern transformer-based methods, like BERT-based and RoBERTa-based tokenizers, have been used in seven and four studies, respectively. The Skip-gram model has also been used in seven studies. The sentence-transformers model (All-mpnet-base-v2) was used in two studies. Two studies also selected textual features manually. All other pre-processing techniques were used in a single study.

4.4 RQ4: Which evaluation approaches have been used?

Table 9 presents the evaluation approaches used in the reviewed studies for classifying issue reports. The most common strategy observed is testing on a fixed set, utilized in 27 studies. Cross-validation techniques also feature prominently, with 10-fold cross-validation appearing in eight studies and 5-fold cross-validation in five studies. Additionally, hybrid approaches, such as time split (sorting issue reports data chronologically before splitting data into train and test sets) with 10-fold cross-validation and time split with testing on a fixed set, were used in three and two studies, respectively. Stratified 10-fold cross-validation was used in a single study.

4.5 RQ5: Which evaluation metrics have been used?

Table 10 summarizes the evaluation metrics used in the literature for classifying issue reports. F1-score is the most widely used metric overall, appearing in 40 studies,

Table 8 Pre-processing techniques used for issue report classification

Pre-processing technique	Relevant studies	Count
TF-IDF	PS1, PS2, PS5, PS6, PS7, PS10, PS13, PS14, PS16, PS26, PS30, PS38, PS42	13
BERT-based tokenizer	PS3, PS12, PS18, PS27, PS31, PS43, PS45	7
Skip-gram model	PS19, PS21, PS22, PS25, PS32, PS39, PS41	7
RoBERTa-based tokenizer	PS11, PS17, PS28, PS46	4
Embedding layer	PS4, PS21, PS33	3
Manual selection of textual features	PS9, PS23	2
All-mpnet-base-v2	PS29, PS35	2
AutoTokenizer from transformers	PS34	1
Text-embedding-ada-002	PS36	1
T5-based tokenizer	PS40	1
GPT-based tokenizer	PS8	1
LDA	PS20	1
GloVe	PS21	1

Table 9 Evaluation approaches used for issue report classification

Evaluation strategy	Relevant studies	Count
Test on a fixed set	PS1, PS2, PS3, PS4, PS5, PS7, PS8, PS11, PS12, PS13, PS15, PS17, PS18, PS23, PS28, PS29, PS30, PS31, PS33, PS33, PS35, PS38, PS40, PS41, PS43, PS45, PS46	27
10-fold CV	PS13, PS20, PS22, PS24, PS37, PS39, PS42, PS44	8
5-fold CV	PS9, PS21, PS27, PS10, PS36	5
Time split with 10-fold CV	PS16, PS26, PS32	3
Time split with test on a fixed set	PS19, PS25	2
Stratified 10-fold CV	PS6	1

Table 10 Evaluation metrics used for issue report classification

Evaluation metrics	Relevant studies	Count
F1-score	PS2, PS4, PS5, PS6, PS7, PS8, PS10, PS11, PS12, PS13, PS15, PS16, PS17, PS18, PS19, PS20, PS21, PS22, PS24, PS25, PS26, PS27, PS28, PS29, PS30, PS31, PS32, PS33, PS34, PS35, PS36, PS37, PS39, PS40, PS41, PS42, PS43, PS44, PS45, PS46	40
Precision	PS1, PS2, PS4, PS5, PS7, PS8, PS11, PS12, PS14, PS15, PS16, PS17, PS18, PS19, PS21, PS22, PS24, PS25, PS26, PS27, PS28, PS29, PS30, PS31, PS32, PS33, PS34, PS35, PS36, PS40, PS42, PS43, PS44, PS45, PS46	35
Recall	PS1, PS2, PS4, PS5, PS7, PS8, PS11, PS12, PS14, PS15, PS16, PS17, PS18, PS19, PS21, PS22, PS24, PS25, PS26, PS27, PS28, PS29, PS30, PS31, PS32, PS33, PS34, PS35, PS36, PS40, PS42, PS43, PS44, PS45, PS46	35
Micro average F1-score	PS3, PS7, PS8, PS12, PS17, PS28, PS29, PS31, PS34, PS35, PS40, PS45, PS46	13
Micro average recall	PS7, PS8, PS12, PS17, PS28, PS29, PS31, PS34, PS40, PS45, PS46	11
Micro average precision	PS7, PS8, PS12, PS17, PS28, PS29, PS31, PS34, PS40, PS45, PS46	11
Accuracy	PS6, PS9, PS14, PS23, PS32, PS36, PS38, PS41	8
AUC	PS16, PS25, PS26, PS32, PS37, PS39	6
MCC	PS6, PS26, PS32, PS37, PS39	5
G-measure	PS37, PS39	2
Hamming Loss	PS11	1

followed by precision and recall, each used in 35 studies. Micro F1-score has been used in 13 studies, and micro recall and micro precision in eleven studies each. Accuracy has been used in eight studies. AUC and MCC are less common, reported in six and five studies, respectively. A couple of studies used G-measure. Hamming Loss is the least used, appearing in a single study.

4.6 RQ6: What study contexts have been considered for issue report classification techniques, in terms of study realism and data sources?

Tables 11 and 12 provide an overview of the context in which studies on issue classification were conducted. It highlights that most studies, 42, were based on OSS projects, while only a few, three, were conducted in CSS contexts. The involvement of practitioners in these studies was minimal, with 42 studies reporting no participation from practitioners, and only four studies included them.

Additionally, 41 studies did not consider important adoption factors for ML techniques beyond prediction accuracy, such as explainability, generalizability, maintainability, and scalability (Rana et al. 2014; Paleyes et al. 2022). Only five studies considered these adoption factors for ML techniques, and among those, three focused solely on explainability, which was approached in a limited manner and lacked evaluation. Only two studies thoroughly investigated the adoption factors for ML techniques (Rana et al. 2014; Paleyes et al. 2022).

5 Discussion

In the following subsections, we discuss (a) key lessons learned from this mapping study on issue report classification and (b) potential areas for future research.

5.1 Prediction accuracy of automatic techniques for issue report classification

The results of this mapping study show that the existing literature on issue report classification has evaluated various automatic techniques, including traditional ML and deep learning-based techniques, as well as more advanced large language models. Among these techniques, BERT-based models and their variants have demonstrated the highest performance [PS32, PS34, PS46]. In particular, RoBERTa has emerged as the state-of-the-art model within the BERT family, as reported by Izadi [PS46]. They evaluated RoBERTa using more than 1 million issue reports from GitHub projects (Kallis et al. 2023) and found that it can classify these issue reports with a micro-

Table 11 Context of studies

Study realism	Practitioners involved	Adoption factors considered
OSS: 42	No: 42	No: 41
CSS: 3	Yes: 4	Yes: 2
		Only explainability, limited, no evaluation: 3

Table 12 Data sources and relevant studies

Dataset source	Relevant studies	Count
GitHub	PS3, PS4, PS6, PS7, PS8, PS9, PS11, PS12, PS17, PS18, PS21, PS22, PS28, PS29, PS30, PS31, PS33, PS34, PS35, PS36, PS40, PS42, PS45, PS46	26
Bugzilla	PS1, PS2, PS5, PS6, PS9, PS15, PS16, PS19, PS23, PS25, PS37, PS38, PS39, PS41, PS43, PS44	17
Jira	PS2, PS6, PS13, PS19, PS20, PS23, PS37, PS39	8
CSS data	PS10, PS26, PS32	3
Mantis, Redhat Bugzilla	PS24	1
Twitter, Other forum posts	PS27	1

average F1 score of 0.89 [PS7, PS46]. The potential reason why RoBERTa is better compared to default BERT-based models (vanilla BERT) could be that RoBERTa was trained on a significantly larger and more diverse corpus, with dynamic masking and longer training times, allowing it to capture deeper contextual representations of language. This results in a model that is more sensitive to subtle linguistic cues in issue reports, such as nuanced distinctions between bug-related descriptions and other types of reports, such as feature requests or maintenance tasks. These improvements are especially beneficial in domains where technical jargon and varied writing styles are common, as is often the case in software issue tracking systems.

Furthermore, the findings from the literature indicate that traditional ML techniques such as logistic regression, SGD-based classifier, and SVM can achieve performance comparable to that of complex deep learning techniques and BERT-based models [PS7, PS16, PS32]. For example, Laiq [PS7] trained an SGD-based classifier on more than 1 million issue reports from GitHub projects (Kallis et al. 2023) and found that it can classify issue reports with a micro-average F1 score of 0.85. The potential reasons for these simple techniques, such as logistic regression, performing well in classifying issue reports could be that the textual data of issue reports are highly redundant and there exist considerably good recurring patterns. In such cases, simple bag-of-words or TF-IDF features can be sufficient for classification, allowing a lightweight model like logistic regression to achieve results comparable to deep learning models.

Lessons learned:

1. BERT-based models (e.g., RoBERTa) are state-of-the-art models for issue report classification [PS32, PS34, PS46].
2. Traditional ML techniques, such as logistic regression, SGD-based classifier, and SVM, can achieve prediction performance comparable to complex deep learning techniques and BERT-based models for the classification of issues reports [PS7, PS16, PS32].

5.2 Lack of consideration of potentially relevant adoption factors

We observe that in the existing literature on issue report classification, there is generally a lack of consideration of other potentially relevant adoption factors beyond prediction accuracy. Other factors of concern for practical adoption are, for example, maintenance cost, scalability, explainability, and adaptability (Rana et al. 2014; Paleyes et al. 2022). These factors do not always complement each other and may require a trade-off. For example, while one technique may be *more accurate* than another, it may score lower in terms of *explainability*.

Approximately 89% of the studies (41 out of 46) evaluated the proposed techniques solely based on prediction accuracy. Only five studies took into account the factors beyond prediction accuracy that may influence the adoption of these techniques in practice. Among those, three focused exclusively on explainability; however, their approach was limited and lacked real evaluation. Only two studies [PS26, PS32] thoroughly studied the adoption factors for ML techniques. These two studies emphasize that practitioners view these factors as important when deciding whether to adopt an ML technique in their practices. These findings also align with the existing literature that investigates factors influencing the adoption of ML techniques in practice (Rana et al. 2014; Paleyes et al. 2022).

Lessons learned and call for future work:

In the existing literature on issue report classification, there is generally a lack of consideration of potentially relevant other adoption factors beyond prediction accuracy. Therefore, future research should: a) take these factors into account when investigating issue report classification within a specific context and b) investigate how factors such as explainability, scalability, and generalizability, beyond just prediction accuracy, affect the practical adoption of ML techniques for automatic issue report classification.

5.3 Lack of industrial validations and involvement of practitioners

We observe that existing research on issue report classification primarily uses archival data from open-source repositories. Approximately 93% (43/46) of the studies used archival data. In contrast, only a small number, 3 out of 46, have used proprietary or industrial data. It is important to evaluate these techniques in real industrial settings to assess their performance, generalizability in other contexts, and practical applicability.

Furthermore, the involvement of practitioners in these studies is notably scarce. Approximately 91% (42/46) of the studies did not involve practitioners. The involvement of practitioners is essential for gathering feedback on the proposed solutions regarding their practical relevance and applicability.

Lessons learned and call for future work:

In the existing literature on issue report classification, in general, there is a lack of industrial validations and involvement of practitioners. Therefore, future work should consider the involvement of practitioners and the evaluation of these techniques in real industrial settings.

5.4 Need to use robust evaluation metrics and approaches

The data from the issue reports is often unbalanced (Laiq et al. 2023; Fan et al. 2018). Therefore, it is important to use evaluation metrics that are robust to data imbalance, such as AUC and MCC, when assessing automatic techniques. However, we have observed a lack of these metrics in the literature. Only 6 of 46 studies have used AUC, and only 5 of 46 have used MCC. This underutilization of robust metrics can lead to misleading conclusions about the effectiveness of proposed techniques. For example, Yao and Shepperd (2021) reanalyzed 12,471 pairwise comparisons of defect prediction models and found that 21.95% of conclusions drawn using F1 were reversed when MCC was applied instead, showing that some methods previously reported as superior were, in fact, worse when evaluated using a balanced metric. Similarly, Huang and Ling (2005) empirically showed that AUC is a better measure than accuracy.

Similarly, we observe that robust evaluation approaches, such as 10-fold cross-validation, are lacking in the reviewed studies. The majority (29 out of 46) rely solely on a fixed test set for performance evaluation. In contrast, only 17 studies employ cross-validation techniques, which are generally considered more reliable to assess the performance of ML techniques. Relying on a single fixed test set increases the risk of biased performance estimates. For example, Ghasemzadeh et al. (2024) found that single hold-out evaluations often overestimated accuracy compared to nested ten-fold cross-validation, leading to increased confidence in the generalizability of a model. They reported that the statistical confidence in the model, based on nested k-fold cross-validation, was four times higher than the confidence obtained from the single holdout-based model.

Table 13 shows recommendations (best practices) for evaluation metrics and approaches that future studies on issue report classification should follow.

Lessons learned and call for future work:

In the existing literature on issue report classification, there is a lack of the use of robust evaluation metrics and approaches. Thus, future work on issue report classification should consider using more robust evaluation metrics and evaluation approaches such as AUC, MCC, and 10-fold CV.

Table 13 Recommendations for evaluation metrics and approaches

Aspect	Current gap	Recommendations
Evaluation metrics	Predominant use of F1-score, Precision, and Recall; low adoption of robust metrics like AUC (6/46) and MCC (5/46).	Use AUC and MCC alongside F1-score and accuracy. Rationale: These metrics are robust to data imbalance and prevent misleading performance conclusions (Huang and Ling 2005; Yao and Shepperd 2021).
Data imbalance	Limited consideration of data imbalance in reported results. Only 15 out of 46 studies discussed this issue, and only nine studies handled the data imbalance issue.	Report class distribution and apply techniques like resampling or cost-sensitive learning. Rationale: Ensures fair evaluation of models on minority classes.
Evaluation approaches	Majority (29/46) rely on a single fixed test set for performance evaluation.	Adopt cross-validation approaches, such as nested k-fold, for more reliable performance estimation. Rationale: Reduces bias and increases confidence in results (Ghasemzadeh et al. 2024).

5.5 Reliability of ground truth used for issue report classification

An important but often overlooked issue in software engineering research is the reliability of ground truth (training data) used to train ML models (Tüzün et al. 2021). This concern is equally relevant for automatic techniques used for issue report classification. Accurate labeling of issue reports, such as identifying them as bugs, features, questions, or documentation tasks, is essential for both training and evaluating ML models. Our analysis of 46 primary studies showed that fourteen studies (approx. 30%) relied exclusively on manually annotated datasets. Five studies (approx. 11%) used a combination of manually annotated and unannotated datasets. Ten studies (approx. 22%) utilized previously known datasets from prior research that were not manually annotated, while seventeen studies (approx. 37%) relied entirely on unannotated datasets. Overall, when combining studies that used either known datasets or completely unannotated data, approximately 59% of the included studies lacked manually annotated ground truth. In addition, we found that data size alone may not be a determining factor in achieving better performance. Our analysis of the 46 primary studies revealed that nine of the smaller datasets achieved better results than their larger counterparts. In five cases, the performance was comparable, and in another five cases, the larger datasets yielded better results.

Several studies (Herzig et al. 2013; Wu et al. 2021; Colavito et al. 2023) have reported that these labels can be noisy, inconsistent, or incorrect. Herzig et al. (2013) manually reviewed more than 7k issue reports and reported that approximately 33.8% of the reports labeled as bugs were actually misclassified, including features, improvements, and documentation tasks. Similarly, Colavito et al. (2022) conducted an error analysis of the issue reports data (Kallis et al. 2023) and reported the presence of noisy labels. ML models trained on such data will produce biased results and can lead to suboptimal decisions in practice.

To address the above-mentioned issue, Colavito et al. (2022) constructed a new small data set by manually reviewing issue reports involving multiple reviewers. In the next step, they evaluated a few-shot learning-based framework, SETFIT, on the small data set and reported that it could improve performance compared to the models trained on noisy labels. These findings emphasize the effectiveness of incorporating small, high-quality labeled datasets to overcome labeling inconsistencies. Despite the proven impact of noisy ground truth on model accuracy and validity, very few studies explicitly address the quality of issue reports classification data in the current literature.

A promising approach to generating annotated data could be the use of LLMs, given their capabilities, for example, to understand and generate code, process natural language, and automate various software engineering tasks. This could be done fully automatically or in a human-in-the-loop setting where LLMs assist human annotators by pre-labeling, suggesting revisions, or highlighting uncertain cases. For example, Colavito et al. (2024) showed that GPT-like models can label issue reports in high agreement with human annotators (Cohen's kappa > 0.7) in the absence of labeled data. Such findings suggest that LLMs can substantially reduce the time and cost associated with manual annotation. However, LLMs have their own challenges, for example, LLMs may hallucinate, propagate biases present in their training data, or exhibit sensitivity to prompt formulation and context. Consequently, we do not advocate fully automatic labeling without verification. In line with previous work (Colavito et al. 2024; Wang et al. 2021), we recommend using LLMs as part of an annotation team. In this setup, LLMs can, for example, pre-annotate data or resolve straightforward cases, while human experts validate complex or ambiguous instances, thereby balancing scalability with reliability.

Lessons learned and call for future work:

1. Issue report classification data (ground truth) can be noisy; we found that around 59% of primary studies have not used manually annotated data. Therefore, future work on issue report classification should carefully consider the reliability of the data for training and evaluating ML models.
2. Few-shot learning-based approaches (e.g., SetFIT) can help tackle the reliability issue of ground truth data (Colavito et al. 2023), i.e., they can effectively classify issue reports using relatively small manually created reliable data sets.

5.6 Practical impact and adoption challenges of automated techniques for issue report classification

The results of this study indicate that while several investigations have evaluated automated techniques for classifying issue reports, there is limited research on the effectiveness of these techniques in real industrial settings. A recent study by Laiq et al. (2024) offers valuable insights from industrial validation in practical settings, demonstrating that automated classification can significantly help triage processes in large-scale software development. However, their study also points out essential adoption-related challenges. For example, they report that practitioners' feedback indicates that classification models need to be explainable to build trust, must integrate seamlessly with existing toolchains, and should adapt to changing contexts to handle concept drift.

Moreover, Laiq et al. (2024) noted that to improve the likelihood of adoption, researchers should go beyond accuracy-focused evaluations using archival data and address practical aspects that practitioners consider important when deciding to adopt automated techniques (Rana et al. 2014; Paleyes et al. 2022). While the accuracy of the models is essential for the technical effectiveness of classifiers, Laiq et al. (2024) emphasize that it alone is insufficient. Their industrial study demonstrated that even when a model achieved strong predictive performance on historical data, practitioners remained skeptical due to concerns about trust, false positives, integration challenges, and model degradation over time. Their findings also highlight that explainability, usability, and maintainability are equally important for adoption. If these aspects are not addressed, classifiers may be technically accurate, but fail to deliver value in real-world workflows. Therefore, evaluations must consider factors such as how easily the tool fits existing processes, how transparent and interpretable its predictions are, and how resilient it is to evolving project contexts.

To systematically address adoption factors and to guide the deployment of their classification tool in an industrial setting, Laiq et al. (2024) used a technology transfer model (Gorschek et al. 2006). This model provides a structured and iterative process for moving research innovations into practice through stages of validation, feedback, and gradual scaling. By following this approach, the researchers were able to involve practitioners from the outset, conduct focus groups to elicit concerns, and conduct initial pilot deployments, iteratively refining the tool based on feedback. The technology transfer model thus helps ensure that technical, organizational, and human factors are addressed in parallel, building confidence and buy-in at each stage. This structured method is especially important for tools such as automated issue classifiers, which can affect core development workflows and require trust to be effective. Therefore, for future studies, we recommend using similar approaches to improve the likelihood of adopting an automated technique in practice for classifying issue reports.

5.7 Issue report classification in the era of large language models

Recent research indicates that strong generative LLMs can often match the performance of fine-tuned encoders in issue labeling, without requiring project-specific training. For example, Colavito et al. (2024) report that GPT-3.5 achieves competitive zero-shot results compared to fine-tuned BERT-style baselines in labeling GitHub issues. These findings

suggest that cross-project generalization, which has traditionally been a challenge for traditional models, may now be possible with promptable models, particularly when combined with light prompt engineering or a few-shot examples. In addition to static classification, LLMs (e.g., GPT-5) can facilitate interactive triage by asking clarifying questions, extracting structured fields, and suggesting assignees. This implies that LLM-centric pipelines could be promising for issue routing in industrial settings.

At the same time, using LLM-based classification involves practical trade-offs. Relying on API-based usage raises concerns about cost and privacy, while closed models can change over time, impacting reported metrics' reproducibility and long-term stability. The performance in zero or few-shot scenarios is sensitive to how prompts are designed and worded, so evaluations need to ensure that prompts are reliable and robust to small changes in phrasing. Furthermore, benchmarks should continuously track the recency and origin of issues to prevent hidden data leakage and accurately measure true generalization. To address cost and improve reliability, hybrid designs, such as utilizing LLM embeddings alongside lightweight classifiers, or employing retrieval-augmented prompts based on project documentation, can help mitigate costs and enhance determinism, while maintaining a high accuracy level.

Looking ahead, we see three concrete research directions. (1) Few-shot and active prompting: develop a budgeted selection of in-context examples drawn from a project's history, with explicit comparisons to fine-tuning and encoder baselines; report cost/latency alongside accuracy. (2) Interactive triage loops: evaluate LLMs that ask for targeted follow-ups, summarize ambiguous reports, and propose labels/assignees with calibrated confidence, using human-in-the-loop protocols and task-time/overrides as primary outcomes. (3) Evaluation standards: Besides standard metrics such as F1, report prompt templates, model version/date, stability across re-runs, privacy posture, and failure analyses (e.g., systematic mislabels, nondeterministic outputs). Overall, we believe that BERT-based fine-tuning remains a strong baseline in data-rich, privacy-constrained settings, while GPT-4-class (and newer models) models enable data-sparse, cross-project, and interactive workflows that traditional pipelines struggle to match.

6 Conclusion and future work

In this study, we provide a comprehensive map of the automatic techniques for issue report classification. Our results indicate that the existing literature applies a broad range of solutions for issue report classification, including traditional ML and deep learning-based techniques and more advanced large language models. As reported in the literature, BERT-based models (e.g., RoBERTa) are state-of-the-art models for issue report classification. In addition, traditional ML techniques, such as logistic regression, SGD-based classifiers, and SVMs, can achieve prediction performance comparable to complex deep learning techniques and BERT-based models for the classification of issue reports.

We found that these studies lack practitioner involvement and fail to consider other potentially relevant adoption factors beyond prediction accuracy, such as the techniques explainability, scalability, and generalizability. In addition, the literature's main focus is on archival data from open-source repositories. Therefore, future

research should address realistic/industrial settings, consider other potentially relevant adoption factors, and actively involve practitioners.

This study focused on automatic techniques for classifying issue reports into bugs and non-bugs (e.g., questions, feature requests, and documentation issues). In future work, we will explore the potential of diverse labeling methods (e.g., skills, time to solve, and GFI) to support issue triage. These labeling methods are essential for a thorough triage process.

A List of included primary studies

Table 14 List of included primary studies

Id	Title	Ref.
PS1	A bug rule based technique with feedback for classifying bug reports	(Zhang and Lee 2011)
PS2	A Comparative Study of Short Text Classification Methods for Bug Report Type Identification	(Polpinij et al. 2022)
PS3	A Comparison of Pretrained Models for Classifying Issue Reports	(Heo et al. 2024)
PS4	A Multimodal Deep Learning Model Using Text, Image, and Code Data for Improving Issue Classification Tasks	(Kwak et al. 2023)
PS5	A Supervised Bug Report Classification with Incorporate and Textual field Knowledge	(Kukkar and Mohana 2018)
PS6	An empirical evaluation of stacked generalization models for binary bug report classification	(Alraddadi and Alshayeb 2024)
PS7	An Intelligent Tool for Classifying Issue Reports	(Laiq 2023)
PS8	Applying Large Language Models to Issue Classification	(Aracena et al. 2024)
PS9	Automated classification of software bug reports	(Otoom et al. 2019)
PS10	Automated classification of unstructured bilingual software bug reports: An industrial case study research	(Köksal and Tekinerdogan 2021)
PS11	Automatic Issue Classifier: A Transfer Learning Framework for Classifying Issue Reports	(Nadeem et al. 2021)
PS12	BERT-Based GitHub Issue Report Classification	(Siddiq and Santos 2022)
PS13	Bug or not? Bug Report classification using N-gram IDF	(Terdchanakul et al. 2017)
PS14	Bug report classification based on vector space model	(Chen et al. 2019)
PS15	Categorizing bugs with social networks: A case study on four open source software communities	(Zanetti et al. 2013)
PS16	Chaff from the Wheat: Characterizing and Determining Valid Bug Reports	(Fan et al. 2018)
PS17	ClassifAI: Automating Issue Reports Classification using Pre-Trained BERT (Bidirectional Encoder Representations from Transformers) Language Models	(Alam et al. 2024)
PS18	Classification of open source software bug report based on transfer learning	(Zhifang et al. 2024)
PS19	Classifying bug reports into bugs and non-bugs using LSTM	(Qin and Sun 2018)
PS20	Classifying bug reports to bugs and other requests using topic modeling	(Pingclasai et al. 2013)
PS21	Classifying issue reports according to feature descriptions in a user manual based on a deep learning model	(Cho et al. 2022)
PS22	Classifying Issues into Custom Labels in GitBot	(Park et al. 2022)

Table 14 (continued)

Id	Title	Ref.
PS23	Classifying software issue reports through association mining	(Zolkeply and Shao 2019)
PS24	Combining text mining and data mining for bug report classification	(Zhou et al. 2016)
PS25	Deep learning based valid bug reports determination and explanation	(He et al. 2020)
PS26	Early Identification of Invalid Bug Reports in Industrial Settings – A Case Study	(Laiq et al. 2022)
PS27	Evaluating software user feedback classifier performance on unseen apps, datasets, and metadata	(Devine et al. 2023)
PS28	Few-Shot Issue Report Classification with Adapters	(Ebrahim and Joy 2024)
PS29	Few-Shot Learning for Issue Report Classification	(Colavito et al. 2023)
PS30	GitHub Bug Classification Using Pipeline Approach in Machine Learning	(Kumar et al. 2023)
PS31	GitHub Issue Classification Using BERT-Style Models	(Bharadwaj and Kadam 2022)
PS32	Industrial adoption of machine learning techniques for early identification of invalid bug reports	(Laiq et al. 2024)
PS33	Issue report classification using a multimodal deep learning technique	(Kwak and Lee 2022)
PS34	Issue Report Classification Using Pre-trained Language Models	(Colavito et al. 2022)
PS35	Large Language Models for Issue Report Classification	(Colavito et al. 2024)
PS36	LLM-BRC: A large language model-based bug report classification framework	(Du et al. 2024)
PS37	Mitigating the impact of mislabeled data on deep predictive models: an empirical study of learning with noise approaches in software engineering tasks	(Shen et al. 2024)
PS38	Optimization of the bug report classification using genetic algorithm	(Gayathrip and Babu 2019)
PS39	Robust Learning of Deep Predictive Models from Noisy and Imbalanced Software Engineering Datasets	(Li et al. 2022)
PS40	Text-To-Text Generation for Issue Report Classification	(Rejithkumar et al. 2024)
PS41	The automatic classification of fault trigger based bug report	(Du et al. 2017)
PS42	Where Is the Road for Issue Reports Classification Based on Text Mining?	(Fan et al. 2017)
PS43	Which bug reports are valid and why? Using the BERT transformer to classify bug reports and explain their validity	(Meng and Visser 2023)
PS44	Works for Me! Cannot Reproduce – A Large Scale Empirical Study of Non-reproducible Bugs	(Rahman et al. 2022)
PS45	Predicting Issue Types with seBERT	(Trautsch and Herbold 2022)
PS46	CatIss: An Intelligent Tool for Categorizing Issues reports using Transformers	(Izadi 2022)

Author Contributions Muhammad Laiq: Conceptualization, Methodology, Validation, Formal analysis, Investigation, Project administration, Data curation, Writing - original draft, Writing - review & editing, Visualization. Felix Dobsław: Conceptualization, Methodology, Validation, Investigation, Project administration, Supervision, Writing - review & editing, Visualization.

Funding Open access funding provided by Mid Sweden University.

Data Availability The data pertaining to this study can be found at https://figshare.com/articles/dataset/SM_S_on_Issue_Report_Classification/31438093.

Declarations

Competing Interests The authors declare no competing interests.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

- Alam, K.A., Jumani, A., Aamir, H., Uzair, M.: Classifai: Automating issue reports classification using pre-trained bert (bidirectional encoder representations from transformers) models. In: Proceedings of the Third ACM/IEEE International Workshop on NL-based Software Engineering, pp. 49–52 (2024)
- Alraddadi, R., Alshayeb, M.: An empirical evaluation of stacked generalization models for binary bug report classification. *Innov. Syst. Softw. Eng.*, 1–16 (2024)
- Ampatzoglou, A., Bibi, S., Avgeriou, P., Verbeek, M., Chatzigeorgiou, A.: Identifying, categorizing and mitigating threats to validity in software engineering secondary studies. *Inf. Softw. Technol.* **106**, 201–230 (2019). <https://doi.org/10.1016/j.infsof.2018.10.006>
- Antoniol, G., Ayari, K., Di Penta, M., Khomh, F., Guéhéneuc, Y.-G.: Is it a bug or an enhancement? a text-based approach to classify change requests. In: Proceedings of the 2008 Conference of the Center for Advanced Studies on Collaborative Research: Meeting of Minds, pp. 304–318 (2008)
- Aracena, G., Luster, K., Santos, F., Steinmacher, I., Gerosa, M.A.: Applying large language models to issue classification. In: Proceedings of the Third ACM/IEEE International Workshop on NL-based Software Engineering, pp. 57–60 (2024)
- Bharadwaj, S., Kadam, T.: Github issue classification using bert-style models. In: Proceedings of the 1st International Workshop on Natural Language-based Software Engineering, pp. 40–43 (2022)
- Borg, M., Jonsson, L., Engström, E., Bartalos, B., Szabó, A.: Adopting automated bug assignment in practice—a longitudinal case study at ericsson. *Empir. Softw. Eng.* **29**(5), 126 (2024)
- Chen, L., Huang, S., Sun, J., Hui, Z., Yang, S.: Bug report classification based on vector space model. *Int. J. Performab. Eng.* **15**(8), 2071 (2019)
- Cho, H., Lee, S., Kang, S.: Classifying issue reports according to feature descriptions in a user manual based on a deep learning model. *Inf. Softw. Technol.* **142**, 106743 (2022)
- Colavito, G., Lanubile, F., Novielli, N.: Issue report classification using pre-trained language models. In: Proceedings of the 1st International Workshop on Natural Language-based Software Engineering, pp. 29–32 (2022)
- Colavito, G., Lanubile, F., Novielli, N.: Few-shot learning for issue report classification. In: 2023 IEEE/ACM 2nd International Workshop on Natural Language-Based Software Engineering (NLBSE), pp. 16–19 (2023). IEEE

- Colavito, G., Lanubile, F., Novielli, N., Quaranta, L.: Large language models for issue report classification (2024)
- Colavito, G., Lanubile, F., Novielli, N., Quaranta, L.: Leveraging gpt-like llms to automate issue labeling. In: Proceedings of the 21st International Conference on Mining Software Repositories, pp. 469–480 (2024)
- Devine, P., Koh, Y.S., Blincoe, K.: Evaluating software user feedback classifier performance on unseen apps, datasets, and metadata. *Empir. Softw. Eng.* **28**(2), 26 (2023)
- Du, X., Zheng, Z., Xiao, G., Yin, B.: The automatic classification of fault trigger based bug report. In: 2017 IEEE International Symposium on Software Reliability Engineering Workshops (ISSREW), pp. 259–265 (2017). IEEE
- Du, X., Liu, Z., Li, C., Ma, X., Li, Y., Wang, X.: Llm-brc: A large language model-based bug report classification framework. *Softw. Qual. J.*, 1–21 (2024)
- Ebrahim, F., Joy, M.: Few-shot issue report classification with adapters. In: Proceedings of the Third ACM/IEEE International Workshop on NL-based Software Engineering, pp. 41–44 (2024)
- Fan, Q., Yu, Y., Yin, G., Wang, T., Wang, H.: Where is the road for issue reports classification based on text mining? In: 2017 ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM), pp. 121–130 (2017). IEEE
- Fan, Y., Xia, X., Lo, D., Hassan, A.E.: Chaff from the wheat: Characterizing and determining valid bug reports. *IEEE Trans. Softw. Eng.* **46**(5), 495–525 (2018)
- GayathriP, M., Babu, G.K.K.: Optimization of the bug report classification using genetic algorithm. (2019). <https://api.semanticscholar.org/CorpusID:212538218>
- Ghasemzadeh, H., Hillman, R.E., Mehta, D.D.: Toward generalizable machine learning models in speech, language, and hearing sciences: Estimating sample size and reducing overfitting. *J. Speech Lang. Hear. Res.* **67**(3), 753–781 (2024)
- Gomes, L.A.F., Silva Torres, R., Côrtes, M.L.: Bug report severity level prediction in open source software: A survey and research opportunities. *Inf. Softw. Technol.* **115**, 58–78 (2019)
- Gorschek, T., Garre, P., Larsson, S., Wohlin, C.: A model for technology transfer in practice. *IEEE Softw.* **23**(6), 88–95 (2006)
- He, J., Xu, L., Fan, Y., Xu, Z., Yan, M., Lei, Y.: Deep learning based valid bug reports determination and explanation. In: 2020 IEEE 31st International Symposium on Software Reliability Engineering (ISSRE), pp. 184–194 (2020). IEEE
- Heo, J., Kwon, G., Kwak, C., Lee, S.: A comparison of pretrained models for classifying issue reports. *IEEE Access* (2024)
- Herzig, K., Just, S., Zeller, A.: It's not a bug, it's a feature: how misclassification impacts bug prediction. In: 2013 35th International Conference on Software Engineering (ICSE), pp. 392–401 (2013). IEEE
- Huang, J., Ling, C.X.: Using auc and accuracy in evaluating learning algorithms. *IEEE Trans. Knowl. Data Eng.* **17**(3), 299–310 (2005)
- Izadi, M.: Catiss: An intelligent tool for categorizing issues reports using transformers. In: Proceedings of the 1st International Workshop on Natural Language-based Software Engineering, pp. 44–47 (2022)
- Kallis, R., Izadi, M., Pascarella, L., Chaparro, O., Rani, P.: The nlbsc'23 tool competition. In: Proceedings of The 2nd International Workshop on Natural Language-based Software Engineering (NLBSE'23) (2023)
- Kim, D., Tao, Y., Kim, S., Zeller, A.: Where should we fix this bug? a two-phase recommendation model. *IEEE Trans. Software Eng.* **39**(11), 1597–1610 (2013)
- Kitchenham, B., Brereton, P., Budgen, D.: Mapping study completeness and reliability-a case study. In: 16th International Conference on Evaluation & Assessment in Software Engineering (EASE 2012), pp. 126–135 (2012). IET
- Kitchenham, B.: What's up with software metrics?-a preliminary mapping study. *J. Syst. Softw.* **83**(1), 37–51 (2010)
- Kitchenham, B.A., Budgen, D., Brereton, P.: Evidence-based Software Engineering and Systematic Reviews. CRC Press, Boca Raton, FL (2016)
- Köksal, Ö., Öztürk, C.E.: A survey on machine learning-based automated software bug report classification. In: 2022 International Symposium on Multidisciplinary Studies and Innovative Technologies (ISMSIT), pp. 635–640 (2022). IEEE
- Köksal, Ö., Tekinerdogan, B.: Automated classification of unstructured bilingual software bug reports: An industrial case study research. *Appl. Sci.* **12**(1), 338 (2021)
- Kukkar, A., Mohana, R.: A supervised bug report classification with incorporate and textual field knowledge. *Proced Comput Sci* **132**, 352–361 (2018)

- Kumar, G.S., Angel, T.S., Chakreborthy, S., Reddy, K.D.: Github bug classification using pipeline approach in machine learning. In: 2023 International Conference on Data Science, Agents & Artificial Intelligence (ICDSAAI), pp. 1–7 (2023). IEEE
- Kwak, C., Lee, S.: Issue report classification using a multimodal deep learning technique (2022)
- Kwak, C., Jung, P., Lee, S.: A multimodal deep learning model using text, image, and code data for improving issue classification tasks. *Appl. Sci.* **13**(16), 9456 (2023)
- Laiq, M., Ali, N.B., Böstler, J., Engström, E.: Early identification of invalid bug reports in industrial settings-a case study. In: International Conference on Product-Focused Software Process Improvement, pp. 497–507 (2022). Springer
- Laiq, M.: An intelligent tool for classifying issue reports. In: 2023 IEEE/ACM 2nd International Workshop on Natural Language-Based Software Engineering (NLBSE), pp. 13–15 (2023). IEEE
- Laiq, M., Ali, N., Böstler, J., Engström, E.: A data-driven approach for understanding invalid bug reports: An industrial case study. *Inf. Softw. Technol.* **164**, 107305 (2023)
- Laiq, M., Ali, N., Böstler, J., Engström, E.: Industrial adoption of machine learning techniques for early identification of invalid bug reports. *Empir. Softw. Eng.* **29**(5), 130 (2024)
- Li, Z., Pan, M., Pei, Y., Zhang, T., Wang, L., Li, X.: Robust learning of deep predictive models from noisy and imbalanced software engineering datasets. In: Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering, pp. 1–13 (2022)
- Meng, Q., Visser, J.: Which bug reports are valid and why? using the bert transformer to classify bug reports and explain their validity. In: Proceedings of the 4th European Symposium on Software Engineering, pp. 52–60 (2023)
- Nadeem, A., Sarwar, M.U., Malik, M.Z.: Automatic issue classifier: A transfer learning framework for classifying issue reports. In: 2021 IEEE International Symposium on Software Reliability Engineering Workshops (ISSREW), pp. 421–426 (2021). IEEE
- Otoom, A.F., Al-jdach, S., Hammad, M.: Automated classification of software bug reports. In: Proceedings of the 9th International Conference on Information Communication and Management, pp. 17–21 (2019)
- Paleyes, A., Urma, R.-G., Lawrence, N.D.: Challenges in deploying machine learning: A survey of case studies. *ACM Comput. Surv.* **55**(6), 1–29 (2022)
- Park, D., Cho, H., Lee, S.: Classifying issues into custom labels in gitbot. In: Proceedings of the Fourth International Workshop on Bots in Software Engineering, pp. 28–32 (2022)
- Parnin, C., Orso, A.: Are automated debugging techniques actually helping programmers? In: Proceedings of the 2011 International Symposium on Software Testing and Analysis, pp. 199–209 (2011)
- Petersen, K.: Measuring and predicting software productivity: A systematic map and review. *Inf. Softw. Technol.* **53**(4), 317–343 (2011)
- Petersen, K., Feldt, R., Mujtaba, S., Mattsson, M.: Systematic mapping studies in software engineering. In: International Conference on Evaluation and Assessment in Software Engineering, pp. 1–10 (2008)
- Petersen, K., Vakkalanka, S., Kuzniarz, L.: Guidelines for conducting systematic mapping studies in software engineering: An update. *Inf. Softw. Technol.* **64**, 1–18 (2015)
- Pingclasai, N., Hata, H., Matsumoto, K.-i.: Classifying bug reports to bugs and other requests using topic modeling. In: 2013 20Th Asia-Pacific Software Engineering Conference (APSEC), vol. 2, pp. 13–18 (2013). IEEE
- Polpinij, J., Kaenampornpan, M., Luaphol, B.: A comparative study of short text classification methods for bug report type identification. In: 2022 Research, Invention, and Innovation Congress: Innovative Electricals and Electronics (RI2C), pp. 27–33 (2022). IEEE
- Qin, H., Sun, X.: Classifying bug reports into bugs and non-bugs using lstm. In: Proceedings of the 10th Asia-Pacific Symposium on Internetwork, pp. 1–4 (2018)
- Rahman, M.M., Khomh, F., Castelluccio, M.: Works for me! cannot reproduce-a large scale empirical study of non-reproducible bugs. *Empir. Softw. Eng.* **27**(5), 111 (2022)
- Rana, R., Staron, M., Hansson, J., Nilsson, M., Meding, W.: A framework for adoption of machine learning in industry for software defect prediction. In: Proceedings of the 9th International Conference on Software Engineering and Applications, pp. 383–392 (2014)
- Rejithkumar, G., Anish, P.R., Ghaisas, S.: Text-to-text generation for issue report classification. In: Proceedings of the Third ACM/IEEE International Workshop on NL-based Software Engineering, pp. 53–56 (2024)
- Shen, J., Li, Z., Lu, Y., Pan, M., Li, X.: Mitigating the impact of mislabeled data on deep predictive models: an empirical study of learning with noise approaches in software engineering tasks. *Autom. Softw. Eng.* **31**(1), 33 (2024)

- Siddiq, M.L., Santos, J.C.: Bert-based github issue report classification. In: Proceedings of the 1st International Workshop on Natural Language-based Software Engineering, pp. 33–36 (2022)
- Terdchanakul, P., Hata, H., Phannachitta, P., Matsumoto, K.: Bug or not? bug report classification using n-gram idf. In: 2017 IEEE International Conference on Software Maintenance and Evolution (ICSME), pp. 534–538 (2017). IEEE
- Trautsch, A., Herbold, S.: Predicting issue types with sebert. In: Proceedings of the 1st International Workshop on Natural Language-based Software Engineering, pp. 37–39 (2022)
- Tüzün, E., Erdogmus, H., Baldassarre, M.T., Felderer, M., Feldt, R., Turhan, B.: Ground-truth deficiencies in software engineering: when codifying the past can be counterproductive. *IEEE Softw.* **39**(3), 85–95 (2021)
- Uddin, J., Ghazali, R., Deris, M.M., Naseem, R., Shah, H.: A survey on bug prioritization. *Artif. Intell. Rev.* **47**, 145–180 (2017)
- Usman, M., Ali, N.B., Wohlin, C.: A quality assessment instrument for systematic literature reviews in software engineering. *E-Inf. Softw. Eng. J.* **17**(1), 230105 (2023). <https://doi.org/10.37190/E-INF230105>
- Wang, S., Liu, Y., Xu, Y., Zhu, C., Zeng, M.: Want to reduce labeling cost? gpt-3 can help. In: Findings of the Association for Computational Linguistics: EMNLP 2021, pp. 4195–4205 (2021)
- Wohlin, C., Runeson, P., Neto, P.A.M.S., Engström, E., Carmo Machado, I., De Almeida, E.S.: On the reliability of mapping studies in software engineering. *J. Syst. Softw.* **86**(10), 2594–2610 (2013)
- Wu, X., Zheng, W., Xia, X., Lo, D.: Data quality matters: A case study on data label correctness for security bug report prediction. *IEEE Trans. Software Eng.* **48**(7), 2541–2556 (2021)
- Yao, J., Shepperd, M.: The impact of using biased performance metrics on software defect prediction research. *Inf. Softw. Technol.* **139**, 106664 (2021)
- Zanetti, M.S., Scholtes, I., Tessone, C.J., Schweitzer, F.: Categorizing bugs with social networks: a case study on four open source software communities. In: 2013 35th International Conference on Software Engineering (ICSE), pp. 1032–1041 (2013). IEEE
- Zhang, T., Lee, B.: A bug rule based technique with feedback for classifying bug reports. In: 2011 IEEE 11th International Conference on Computer and Information Technology, pp. 336–343 (2011). IEEE
- Zhang, J., Wang, X., Hao, D., Xie, B., Zhang, L., Mei, H.: A survey on bug-report analysis. *Sci. China Inf. Sci.* **58**(2), 1–24 (2015)
- Zhang, T., Han, D., Vinayakarao, V., Irsan, I.C., Xu, B., Thung, F., Lo, D., Jiang, L.: Duplicate bug report detection: How far are we? *ACM Trans. Softw. Eng. Methodol.* **32**(4), 1–32 (2023)
- Zhifang, L., Kun, W., Qi, Z., Shengzong, L., Yan, Z., Jianbiao, H.: Classification of open source software bug report based on transfer learning. *Expert. Syst.* **41**(5), 13184 (2024)
- Zhou, Y., Tong, Y., Gu, R., Gall, H.: Combining text mining and data mining for bug report classification. *J. Softw. Evol. Process* **28**(3), 150–176 (2016)
- Zolkeply, M.S., Shao, J.: Classifying software issue reports through association mining. In: Proceedings of the 34th ACM/SIGAPP Symposium on Applied Computing, pp. 1860–1863 (2019)
- Zou, W., Lo, D., Chen, Z., Xia, X., Feng, Y., Xu, B.: How practitioners perceive automated bug report management techniques. *IEEE Trans. Software Eng.* **46**(8), 836–862 (2018)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.